

El Journey empieza a escribirse

Javier G. Grandez

2026-08-17

Tabla de contenidos

Journey	4
El recorrido	4
title: “Cuando el proceso empezó a importar”	5
Lo que quedó	6
Fuentes	6
title: “Guardar antes de interpretar”	7
Una consecuencia inesperada	8
Fuentes	9
title: “Un gráfico que no quería ser un diagrama”	10
Fuentes	12
title: “Primero la especificación, después el código”	13
Fuentes	15
title: “IASI Graphics destapa la metodología”	16
Fuentes	18
title: “Todo se reduce a inputs”	19
Fuentes	21
title: “De metodología escrita a sistema ejecutable”	22
Fuentes	24
title: “Instructions, skills y plataformas”	25
Fuentes	27
title: “El lugar natural del cambio”	28
Lo que quedó	30
Fuentes	30
title: “Cuando OpenSpec dejó de ser necesario”	31
Fuentes	33

title: “Inputs no es lo mismo que entender”	34
Fuentes	36
title: “define no genera: reconcilia”	37
Fuentes	40
title: “IASI decide construirse con IASI”	41
Fuentes	42
title: “Nos perdimos y paramos”	43
Lo que quedó	45
Fuentes	45
title: “El Journey empieza a escribirse”	46
Fuentes	47

Journey

IASI no nació de un diseño cerrado ni de una metodología escrita de principio a fin. Fue apareciendo mientras intentábamos resolver problemas concretos, discutir decisiones, probar herramientas, equivocarnos, corregirnos y descubrir que algunas de las piezas que parecían auxiliares eran, en realidad, parte de la metodología.

Este Journey conserva ese recorrido.

No pretende sustituir a los libros, a las **definitions**/, a los ADR/EDR ni al código. Cada uno de esos artefactos responde a otra pregunta.

- Los **libros** cuentan el conocimiento ya consolidado.
- Las **definitions** expresan lo que IASI entiende y mantiene como estado canónico.
- Las **decisiones** fijan lo que se decidió.
- El **código** muestra lo que se construyó.
- El **Journey** conserva cómo llegamos hasta allí.

Por eso aquí aparecerán ideas que después fueron descartadas, nombres que cambiaron, arquitecturas provisionales, preguntas sin respuesta y momentos en los que hubo que parar.

No corregiremos retrospectivamente una entrada para hacer que el pasado parezca más inteligente de lo que fue. Si algo cambia, la historia continuará en otra entrada.

El recorrido

Los capítulos aparecen en el orden en que el proyecto fue cambiando.

No están agrupados en partes ni reorganizados retrospectivamente por temas. La secuencia es deliberada: cada capítulo hereda las preguntas del anterior y deja abiertas las que empujarán al siguiente.

title: “Cuando el proceso empezó a importar”

Este capítulo forma parte del recorrido de IASI. La narración completa continúa en el documento siguiente.

Al principio, casi todo lo que producíamos tenía una forma conocida: repositorios, libros, documentación, código, decisiones. Eran resultados. Cosas que podían quedar limpias, ordenadas y explicadas después.

Pero el trabajo real no se parecía demasiado a esos resultados.

Las ideas aparecían en una conversación. Una propuesta parecía buena durante diez minutos y luego dejaba de serlo. Un nombre llevaba a otro. Una solución técnica abría una pregunta metodológica que no estaba en el problema original. A veces una frase informal contenía más información sobre la decisión que el documento final que escribíamos después.

Empezó a aparecer una incomodidad: si conservábamos únicamente el resultado, estábamos perdiendo la parte más interesante.

No era un problema de auditoría. Git ya podía decir qué fichero había cambiado. Tampoco era un problema de documentación. Los libros podían explicar perfectamente el estado final.

El problema era otro:

¿Cómo se había llegado hasta allí?

Esa pregunta resultó ser mucho más importante de lo que parecía.

Una metodología suele escribirse desde el final. Cuando todo ha encajado, se eliminan las dudas, se suavizan los giros y se presenta una secuencia limpia. El lector recibe una ruta recta aunque el descubrimiento real haya sido un laberinto.

IASI estaba naciendo precisamente en ese laberinto.

Por eso empezó a tomar forma una separación que después sería fundamental:

Journey	→ cómo pensamos, discutimos y descubrimos
Decisiones	→ qué decidimos
Books	→ qué conocimiento consolidamos
Artefactos	→ qué construimos

No eran cuatro formas de documentar lo mismo. Eran cuatro clases distintas de evidencia.

El Journey tenía además una propiedad que los demás no necesitaban: debía poder estar equivocado.

Si en un momento pensábamos que una arquitectura era correcta, la entrada debía conservarlo. Si dos días después descubríamos que no funcionaba, no se reescribía el pasado. Se añadía el siguiente episodio.

Eso cambia completamente la naturaleza del documento.

El Journey no es una especificación mutable que siempre representa la verdad actual. Es una secuencia de estados de comprensión. Cada entrada dice, en esencia:

Con lo que sabíamos entonces, esto fue lo que vimos.

Y ahí apareció otra idea que terminaría siendo central: las conversaciones debían conservarse como fuente inmutable. La narración podía interpretar. El chat bruto no.

Así, por primera vez, el proceso dejó de ser el residuo de construir IASI y empezó a convertirse en uno de sus productos.

No porque cada discusión merezca ser publicada. Muchas no lo merecen. Sino porque, cuando una idea cambia la arquitectura, la metodología o nuestra manera de trabajar, el camino que la produjo contiene conocimiento que el resultado final ya no puede mostrar.

Lo que quedó

El Journey no documentaría el estado de IASI.

Documentaría **su evolución**.

Y no intentaría demostrar que habíamos tenido razón desde el principio.

Precisamente lo contrario.

Fuentes

- Memoria de trabajo de IASI, 2026-08-13.
- 2026-08-13-iasi-graphics-chat.md.
- Conversaciones sobre la separación entre Journey, decisiones, libros y artefactos.

title: “Guardar antes de interpretar”

Este capítulo forma parte del recorrido de IASI. La narración completa continúa en el documento siguiente.

La primera necesidad práctica del Journey no fue escribir.

Fue **guardar**.

Mientras trabajábamos con ChatGPT, Codex y otras herramientas, las conversaciones empezaban a contener demasiadas decisiones como para confiar en que siempre podríamos reconstruirlas después. Había chats de móvil, chats de ordenador, tareas de Codex y conversaciones que atravesaban varios problemas a la vez.

La reacción inicial fue muy simple:

Quiero bajarme todas las conversaciones.

Parecía una tarea de archivo. Terminó definiendo una arquitectura de conocimiento.

Porque enseguida apareció una distinción:

```
raw  
treated
```

El material **raw** debía conservarse tal como había ocurrido. Sin reescribir frases, sin ordenar retrospectivamente la discusión, sin quitar contradicciones porque ahora resultarían incómodas.

El material **treated**, en cambio, podía evolucionar. Podíamos resumirlo, relacionarlo con otras conversaciones, extraer decisiones, escribir una entrada de Journey o convertir una conclusión en una definition.

La regla emergió casi sola:

Raw nunca cambia. Treated puede evolucionar todas las veces que sea necesario.

Eso resolvía un problema que todavía no habíamos formulado explícitamente: cómo permitir interpretación sin perder evidencia.

Si solo guardábamos los documentos tratados, cualquier narración futura dependería de la última interpretación. Si solo guardábamos los chats, tendríamos un archivo enorme pero poco navegable.

Necesitábamos ambas cosas.

```
conversación original
    ↓
    raw
    ↓
interpretación
    ↓
journey / decisions / definitions / books
```

Esta separación resultó especialmente importante cuando Codex entró de lleno en el trabajo. Sus tareas no eran simplemente otra ventana de ChatGPT. Tenían su propio contexto, sus actualizaciones intermedias y una relación directa con el workspace y los cambios del código.

De pronto el desarrollo de IASI estaba repartido entre personas, agentes y repositorios.

El Journey no podía depender de recordar después qué había dicho quién.

Por eso los chats se convirtieron en una forma de evidencia.

No porque cada línea sea importante. Muchísimas no lo son. Pero la conversación conserva algo que los artefactos finales pierden: **la causalidad**.

Un fichero puede mostrar que añadimos `inputs/`.

El chat puede mostrar por qué, qué alternativas descartamos, qué entendíamos por input en ese momento y qué pregunta provocó el cambio.

Ese es el material que necesita el Journey.

Una consecuencia inesperada

Guardar conversaciones parecía una tarea administrativa.

En realidad estaba definiendo un principio de IASI:

Antes de transformar conocimiento, conserva la fuente.

Más tarde aparecerían `inputs/`, `definitions/`, reconciliación semántica y trazabilidad. En ese momento todavía no teníamos todos esos nombres.

Pero la intuición ya estaba ahí.

Primero conservar.

Después interpretar.

Fuentes

- `2026-08-13-iasi-graphics-chat.md`.
- `trabajar-en-todo-iasi-org.md`.
- Conversaciones de 2026-08-13 sobre exportación de ChatGPT y Codex.

title: “Un gráfico que no quería ser un diagrama”

Este capítulo forma parte del recorrido de IASI. La narración completa continúa en el documento siguiente.

`iasi-graphics` no empezó como un proyecto de lenguaje ni como un experimento metodológico.

Empezó con una molestia visual.

Ya teníamos PlantUML para diagramas técnicos. Funcionaba bien para lo que era. Pero había figuras que pedían otra cosa: composiciones más editoriales, más cercanas a una presentación. Cajas, círculos, relaciones simples, una idea explicada visualmente. No un UML disfrazado.

La primera búsqueda fue instrumental:

¿Con qué hacemos esos dibujos?

Aparecieron Inkscape, Figma, PowerPoint, generación de PPTX, JavaScript. Todos podían resolver una parte, pero chocaban con algo que todavía no habíamos convertido en principio.

Podíamos generar una presentación por código, sí.

Pero entonces el usuario tendría que pensar en el mecanismo gráfico.

Y eso era exactamente lo que no queríamos.

La frase que cambió el problema fue aproximadamente esta:

La trampa la has puesto en el JavaScript.

Habíamos conseguido una imagen declarando algo en apariencia sencillo, pero la complejidad real seguía escondida en un código específico preparado para esa composición. No habíamos creado un sistema. Habíamos movido la dificultad de sitio.

Entonces apareció otra posibilidad:

¿Y si escribimos **qué** queremos representar y el motor decide **cómo** dibujarlo?

Eso abrió un mundo distinto.

Ya no hablábamos de automatizar PowerPoint. Hablábamos de un lenguaje declarativo.

```
intención visual
  ↓
DSL
  ↓
motor
  ↓
SVG / PNG
```

El usuario no debía especificar coordenadas, tamaños, CSS ni primitivas gráficas. Debía expresar conceptos: flujo, comparación, ecosistema, elementos, relaciones, énfasis.

La complejidad gráfica pertenecía al motor.

Esto encajaba con una idea que IASI repetiría después en otros contextos: una buena ingeniería puede ser muy compleja por dentro sin trasladar esa complejidad al consumidor.

La discusión sobre YAML duró poco porque el punto no era YAML. El punto era encontrar una sintaxis que pudiera escribirse, versionarse y, sobre todo, insertarse naturalmente en un documento Quarto.

Incluso la forma de invocarlo importaba:

no queríamos un bloque extraño pegado desde fuera, sino algo que pareciera parte natural del ecosistema de bloques ejecutables.

`iasi-graphics` comenzó así a adquirir una identidad propia:

- entrada textual;
- lenguaje declarativo;
- motor independiente;
- salida gráfica;
- integración con Quarto;
- implementación preferentemente en Go;
- nada de obligar al usuario a escribir JavaScript.

Lo curioso es que todavía creíamos que estábamos diseñando un pequeño artefacto gráfico.

En realidad, acabábamos de crear el problema perfecto para descubrir cómo debía funcionar IASI.

Fuentes

- 2026-08-13-iasi-graphics-chat.md.
- chatgpt-20260813215000-iasi-graphics-codex.md.

title: “Primero la especificación, después el código”

Este capítulo forma parte del recorrido de IASI. La narración completa continúa en el documento siguiente.

Cuando nació `iasi-graphics`, el repositorio estaba prácticamente vacío.

Había una carpeta:

```
inputs/
```

y dentro, documentos que describían lo que queríamos construir.

Eso produjo una escena bastante reveladora.

La conversación empezó a deslizarse hacia comandos de compilación, tests y estructura Go antes de que existiera código. La reacción fue inmediata:

¿Qué coño va a construir en un directorio vacío?

La corrección parecía trivial, pero fijó una manera de trabajar.

No había que darle a Codex una estructura inventada para que empezara a picar código. Había que darle los documentos y pedirle que reconstruyera la intención.

Así surgió el primer experimento serio:

```
inputs/
  ↓
Codex lee
  ↓
explica qué cree que hay que construir
  ↓
identifica arquitectura, vertical slice y ambigüedades
```

Con una condición importante:

no implementar nada todavía.

Queríamos comprobar si la especificación era suficiente para que otro agente entendiera el producto sin que nosotros le sopláramos la respuesta en el prompt.

Eso convertía a Codex en algo más interesante que un generador de código. Se convertía en una prueba de nuestra propia claridad.

Si interpretaba mal el producto, el primer sospechoso no debía ser Codex.

Debían ser los inputs.

La pregunta cambió de:

¿Puede Codex programar esto?

a:

¿Nuestros documentos contienen suficiente información para que alguien distinto de nosotros reconstruya correctamente lo que queremos?

Codex respondió identificando capacidades, arquitectura, un vertical slice y, sobre todo, ambigüedades.

Y ahí ocurrió algo importante.

Las ambigüedades no se corrigieron editando los documentos originales.

Se añadió nueva información.

Los inputs originales eran evidencia de entrada. Las aclaraciones también eran inputs, pero de otra naturaleza.

Todavía no habíamos llegado a **externals/** e **internals/**, pero la necesidad ya estaba ahí.

Esta pequeña prueba introdujo varios principios que después aparecerían formalmente en IASI:

- la implementación no debe comenzar si falta información esencial;
- un agente no debe inventar decisiones para rellenar huecos;
- las ambigüedades detectadas son información útil;
- la fuente original no se modifica para hacer que parezca más completa de lo que era;
- el conocimiento puede crecer por capas.

Y, quizá lo más importante:

La especificación no es un documento que acompaña al software. Debe ser capaz de **producir** software.

No de forma mágica ni determinista. Pero sí con suficiente precisión para que un agente diferente pueda pasar de intención a implementación sin depender de una conversación privada que nadie más puede reconstruir.

`iasi-graphics` fue la primera vez que intentamos comprobarlo en serio.

Fuentes

- `chatgpt-20260813215000-iasi-graphics-codex.md`.

title: “IASI Graphics destapa la metodología”

Este capítulo forma parte del recorrido de IASI. La narración completa continúa en el documento siguiente.

Al día siguiente, `iasi-graphics` dejó de ser solo un proyecto gráfico.

La conversación empezó con una idea técnica: montarlo de forma parecida a PlantUML. Enviar una descripción a un servidor y recibir el resultado.

Eso permitía empujar toda la complejidad al servidor:

```
cliente ligero
  ↓
descripción
  ↓
servidor iasi-graphics
  ↓
SVG o PNG
```

El servidor podía usar Go y cualquier otra herramienta que necesitara. Quarto podía integrarse mediante un filtro Lua. Un agente podía utilizarlo mediante MCP. Incluso un motor de generación de imágenes podría incorporarse detrás sin cambiar el contrato del consumidor.

Era una buena arquitectura de producto.

Pero entonces apareció una observación más importante:

Estamos construyendo en base a especificaciones.

Y casi inmediatamente:

`iasi-graphics` hay que documentarlo en paralelo como lab o como decidamos llamarlo.

La palabra *lab* era provisional. La idea no.

Había tres cosas ocurriendo al mismo tiempo:

1. Estamos construyendo *iasi-graphics*.
2. Estamos descubriendo una metodología mientras lo hacemos.
3. Necesitamos conservar el viaje que explica ese descubrimiento.

iasi-graphics se convirtió así en la PoC de algo que todavía no tenía repositorio completo.

No queríamos escribir primero una metodología teórica y luego buscar un ejemplo que la demostrara. Queríamos que el caso real obligara a la metodología a responder preguntas de verdad.

El primer golpe llegó con *inputs*.

Al principio podía parecer que *inputs/* era simplemente la carpeta donde habíamos puesto nuestras especificaciones.

Pero Javier hizo una precisión decisiva:

Los inputs son los documentos que ya existen, en el formato que sea: caso de negocio, funcional, etc. Son entradas externas.

Eso impedía diseñar IASI suponiendo que la información llegaría ya limpia, estructurada y escrita para la metodología.

Luego apareció una segunda clase de entrada.

Codex había analizado los documentos y había detectado ambigüedades. Nosotros respondíamos con aclaraciones. Esas aclaraciones también alimentaban el proyecto, pero no eran documentos externos.

Así nacieron conceptualmente:

```
inputs/  
  externals/  
  internals/
```

Y poco después apareció una tercera fuente: información obtenida por ingeniería inversa, análisis o trabajo del propio sistema.

```
inputs/  
  externals/  
  internals/  
  obtained/
```

La metodología empezaba a materializarse porque un proyecto real la estaba obligando a distinguir clases de conocimiento.

`iasi-graphics` había dejado de ser solo el primer consumidor.

Se había convertido en el instrumento con el que estábamos descubriendo IASI.

Fuentes

- `chatgpt-20260814084622-iasi-graphics-metodologia.md`.

title: “Todo se reduce a inputs”

Este capítulo forma parte del recorrido de IASI. La narración completa continúa en el documento siguiente.

La palabra `inputs` parecía demasiado sencilla para sostener una parte importante de una metodología.

Precisamente por eso funcionó.

Habíamos empezado distinguiendo documentos externos de aclaraciones internas:

```
inputs/  
  externals/  
  internals/
```

La regla era clara: los externos no se modificaban desde IASI. Si un documento externo estaba incompleto, no se “arreglaba” para facilitar el trabajo. Se añadía un input interno que aclaraba lo necesario.

Esto permitía conservar la diferencia entre:

lo que recibimos

y

lo que añadimos para poder trabajar con ello.

Pero los `internals/` tenían un problema distinto.

Estaban vivos.

Una aclaración podía ser necesaria durante una iteración y dejar de ser válida en la siguiente. Borrarla destruía historia. Mantenerla activa contaminaba el estado actual.

La solución apareció con una palabra:

```
archived/
```

No hacía falta una carpeta **active/**. Lo activo era simplemente lo que estaba en el lugar normal. Lo que dejaba de participar en la interpretación pasaba al histórico.

Después apareció **obtained/**.

Había conocimiento que no venía de fuera ni era una decisión añadida manualmente. Podía surgir de ingeniería inversa, análisis del código existente, inspección de infraestructura o trabajo del propio proceso.

También era entrada para fases posteriores.

Así la estructura quedó:

```
inputs/  
  externals/  
  internals/  
  obtained/
```

con históricos asociados cuando fueran necesarios.

La simplificación produjo una frase reveladora:

Con lo que todo se reduce a un directorio: **inputs**.

No significaba que todo conocimiento fuera igual.

Significaba que, para IASI, todo lo que alimenta el razonamiento entra por un mismo concepto.

El formato tampoco debía imponerse demasiado pronto.

Un input podía ser Markdown, una especificación funcional, un documento de negocio, información obtenida de un sistema o cualquier artefacto que aportara conocimiento.

La metodología no debía exigir que el mundo exterior se adaptara a ella antes de poder empezar a pensar.

Entonces apareció la primera etapa formal:

Valida. ¿Es suficiente?

La pregunta de validación no era si los documentos estaban perfectos.

Era si existía suficiente conocimiento para avanzar sin inventar decisiones importantes.

Y apareció una regla de gate que después crecería mucho:

```
si validate falla  
↓  
no se avanza
```

Incluso un input interno deliberadamente parcial podía permitir avanzar si decía, por ejemplo:

por ahora créame un proyecto Quarto.

La suficiencia dependía de la siguiente fase, no de una fantasía de documentación completa.

Ahí IASI empezó a parecerse menos a una colección de buenas prácticas y más a una máquina de trabajo.

Fuentes

- chatgpt-20260814084622-iasi-graphics-metodologia.md.

title: “De metodología escrita a sistema ejecutable”

Este capítulo forma parte del recorrido de IASI. La narración completa continúa en el documento siguiente.

Hubo un momento en que la distinción se volvió inevitable:

el Volumen III podía **explicar** IASI, pero no podía **ser** IASI.

Una metodología descrita en un libro todavía obliga a cada persona o agente a interpretar cómo aplicarla. Si queríamos que distintos agentes trabajaran de manera consistente, había que materializarla.

La idea apareció alrededor de los comandos.

No comandos de Codex.

No prompts particulares de Copilot.

Comandos de IASI.

```
/validate  
/status  
/work  
/archive  
/obtain
```

Los nombres todavía podían cambiar. Lo importante era quién era dueño de su significado.

IASI.

Eso llevó a separar el sistema conceptual de sus plataformas de ejecución.

```
IASI
  commands/
  skills/
  mcp/
  core/
  schemas/
  adapters/
    codex/
    copilot/
  ...
```

Codex podía soportar una forma de comando. Copilot otra. Mañana aparecería otra herramienta.

La metodología no debía convertirse en una colección de instrucciones escritas para el producto de moda de ese mes.

La dirección correcta de dependencia era la contraria:

```
plataforma
  ↓
adaptador
  ↓
IASI
```

Esta decisión cambió también la función del repositorio `iasi`.

Hasta entonces los libros explicaban la metodología y varios artefactos la apoyaban. Ahora hacía falta un lugar que contuviera **su materialización operativa**.

El repositorio `iasi` debía convertirse en eso.

No necesariamente un único ejecutable monolítico, ni una arquitectura completamente decidida desde el primer día. Pero sí la fuente canónica de:

- comandos;
- skills;
- instrucciones;
- validaciones;
- estructura de conocimiento;
- integración agentic;
- runtime cuando fuera necesario.

La diferencia empezó a formularse así:

El Volumen III explica y define la metodología.
`iasi` la implementa.

Y `iasi-graphics` seguía desempeñando un papel crucial.

Era el proyecto que obligaba a comprobar si esa materialización servía para algo.

La metodología no iba a escribirse en una torre de marfil y después aplicarse.

Iba a ser descubierta bajo carga.

Fuentes

- `chatgpt-20260814084622-iasi-graphics-metodologia.md`.
- `chatgpt-20260814104100-iasi-comandos-skills-mcp.md`.

title: “Instructions, skills y plataformas”

Este capítulo forma parte del recorrido de IASI. La narración completa continúa en el documento siguiente.

La palabra *agentic* empezó a quedarse pequeña casi en cuanto intentamos usarla.

Sabíamos que había algo que describía cómo debía trabajar un agente. También había operaciones que el humano podía solicitar, procedimientos especializados y herramientas externas.

Meterlo todo bajo una sola carpeta habría funcionado durante unos días.

Después habría empezado la confusión.

La discusión con Codex y Copilot ayudó a separar las piezas:

Instructions	→ cómo debe comportarse el agente
Skills	→ cómo se hace bien una clase de tarea
Commands	→ qué tarea se solicita ahora
Tools	→ qué capacidad ejecutable puede utilizar
MCP	→ cómo accede a capacidades externas
Agent	→ quién razona y ejecuta

La distinción más importante no era terminológica.

Era de propiedad.

Cuando preguntamos si Codex soportaba skills, apareció una tentación evidente: diseñar las skills directamente para Codex.

Pero eso habría invertido la arquitectura.

La conclusión fue:

Una skill IASI no es una skill de Codex.

Codex puede tener un adaptador capaz de representar una skill IASI en su formato. Copilot puede necesitar otro. Otra plataforma futura podrá utilizar una tercera forma.

```
IASI skill
  adapter Codex
  adapter Copilot
  ...
```

La skill es producto.

La plataforma es entorno de ejecución.

Poco después afinamos otra categoría.

No era **agentics**/ lo que buscábamos para las reglas generales.

Era:

```
instructions/
```

Ahí debían vivir cosas como:

- no inventar información ausente;
- validar antes de modificar;
- respetar el alcance;
- tratar explícitamente la incertidumbre;
- normas de escritura cuando se generan documentos;
- límites de decisión del agente.

Eso permitió evitar un error habitual: esconder reglas globales dentro de cada prompt.

Si una norma forma parte de IASI, debe existir una vez en IASI.

Después vendría el problema de instalación.

Una metodología operativa necesita llegar al proyecto donde se utiliza.

La comparación con OpenSpec ayudó:

```
iasi
  ↓ install
proyecto / workspace
```

Pero también aquí apareció una diferencia útil.

En un proyecto concreto podía instalarse únicamente lo necesario. En un workspace genérico, donde todavía no sabemos qué trabajo aparecerá, tenía sentido instalar el conjunto completo.

Todavía estábamos resolviendo mecánica, CLI y rutas.

Pero la arquitectura conceptual ya había quedado bastante limpia:

IASI define el comportamiento y las capacidades.

Las plataformas se adaptan a IASI, no IASI a las plataformas.

Fuentes

- `chatgpt-20260815133533-iasi-agentics-skills-codex.md`.
- `chatgpt-20260815135800-iasi-agentics-instructions-cli.md`.

title: “El lugar natural del cambio”

Este capítulo forma parte del recorrido de IASI. La narración completa continúa en el documento siguiente.

La discusión empezó con una nueva necesidad.

Una de esas frases inocentes que, en software, pueden terminar creando una capa, un parámetro, una excepción y tres años de mantenimiento.

La reacción habitual es preguntar:

¿Dónde metemos esto?

Pero después de años diseñando y revisando sistemas, Javier planteó otra vara de medir:

Cuando aparece una nueva necesidad, debe ir a su sitio natural, que además es intuitivo. Si no es así, tu sistema está mal.

La frase tenía más profundidad de la que aparentaba.

Un sistema no es extensible porque tenga muchos puntos de extensión. Es extensible cuando las nuevas variantes encuentran un lugar que parece haber estado esperándolas, aunque nadie conociera esa variante cuando se diseñó la arquitectura.

El ejemplo bancario lo hizo concreto.

En un sistema de préstamos y créditos existían numerosos productos numéricos. Para incorporar una nueva forma no era necesario modificar el núcleo y añadir condiciones.

Se creaba un módulo con una nomenclatura asociada al producto.

El sistema lo descubría.

Nada más.

Eso llevó a una formulación mucho más precisa:

No diseñaste los productos futuros. Diseñaste el lugar natural donde los productos futuros podrían existir.

No se trata, por tanto, de predecir todas las necesidades.

Se trata de identificar correctamente los ejes de variación.

La conversación había empezado por otra observación: muchas arquitecturas envejecen acumulando excepciones.

Cada una puede haber sido razonable.

El problema está en la serie:

```
nueva necesidad
  ↓
pequeña excepción
  ↓
otra necesidad
  ↓
otra excepción
  ↓
otra capa
```

Al final, parte de lo que llamamos complejidad *legacy* no representa la complejidad del dominio.

Representa el coste histórico de no haber podido rehacer soluciones anteriores.

Y aquí entraron los Sistemas Inteligentes.

Históricamente, un arquitecto podía detectar que una abstracción ya no era correcta y aun así decidir no tocarla. Comprender todas las dependencias, decisiones antiguas, efectos laterales y riesgos podía costar demasiado.

Si un Sistema Inteligente reduce radicalmente el coste de comprender un sistema, cambia la economía de la refactorización.

La pregunta deja de ser:

¿Cómo añadimos la sexta excepción sin romper nada?

y puede convertirse en:

¿Sigue siendo correcta esta abstracción? ¿Dónde estaría ahora el lugar natural de esta necesidad?

Esto encajó con otra tesis central de IASI:

Si la implementación cada vez cuesta menos, el pensamiento cada vez vale más.

La IA no cambia la ingeniería únicamente porque escriba código más deprisa.

Puede cambiarla porque hace económicamente posible **volver a pensar** sistemas que antes solo podíamos seguir parcheando.

Lo que quedó

Una buena arquitectura no adivina el futuro.

Hace espacio para el cambio.

Y cuando lo nuevo deja de encontrar su sitio natural, la primera reacción no debería ser inventarle uno.

Debería ser cuestionar el modelo.

Fuentes

- `chatgpt-20260816001600-tenemos-una-nueva-necesidad-refactorizacion.md`.
- `chatgpt-20260816003400-arquitectura-refactorizacion-lugar-natural-del-cambio.md`.
- Memoria de IASI sobre el principio «Todo se debe cuestionar».

title: “Cuando OpenSpec dejó de ser necesario”

Este capítulo forma parte del recorrido de IASI. La narración completa continúa en el documento siguiente.

OpenSpec llevaba un tiempo formando parte del paisaje mental de IASI.

No necesariamente como dependencia definitiva, pero sí como una pieza razonable entre los documentos de entrada y la implementación.

Hasta que apareció una pregunta muy pequeña:

Tenemos nuestros inputs. ¿Para qué necesitamos OpenSpec, al menos por ahora?

La pregunta hizo algo muy IASI: obligó a justificar una pieza que ya habíamos dado por incorporada.

El dibujo inicial podía expresarse así:

```
conocimiento
  ↓
inputs
  ↓
OpenSpec
  ↓
agente
  ↓
implementación
```

Pero IASI había evolucionado.

Los **inputs/** ya eran legibles por humanos y agentes, versionables, independientes de plataforma y suficientemente flexibles para recibir conocimiento sin obligar a convertirlo primero a otro formato.

Así que la pregunta correcta pasó a ser:

¿Qué problema real resuelve hoy OpenSpec que IASI no pueda resolver sin él?

La respuesta fue incómodamente sencilla:

ninguno imprescindible.

Eso no convirtió OpenSpec en una mala herramienta.

Simplemente convirtió su incorporación actual en arquitectura anticipada.

`inputs → OpenSpec → agente`

añadía una transformación que todavía no necesitábamos.

Podía volver a tener sentido más adelante: cientos de documentos, trazabilidad formal compleja, cobertura, gestión estricta de cambios, interoperabilidad con un ecosistema que justificara el coste.

Pero IASI no debía construir esa infraestructura por si acaso.

Esta decisión fue importante por una razón que trascendía OpenSpec.

Estábamos empezando a aplicar a IASI su propio principio:

Todo se debe cuestionar.

No preguntar:

¿Cómo integramos OpenSpec?

sino:

¿Qué necesitamos para resolver el problema?

Y solo después comprobar si una herramienta existente satisface esa necesidad.

La retirada provisional de OpenSpec simplificó el flujo y dejó al descubierto el siguiente problema real.

Si los inputs alimentaban directamente a IASI, necesitábamos saber si eran suficientes y coherentes para avanzar.

Así apareció con claridad `/validate`.

La eliminación de una pieza no dejó un hueco.

Reveló la pieza que realmente necesitábamos.

Fuentes

- `chatgpt-20260817130542-iasi-workflow-inputs-definitions.md`.

title: “Inputs no es lo mismo que entender”

Este capítulo forma parte del recorrido de IASI. La narración completa continúa en el documento siguiente.

Durante varios días **inputs/** había demostrado ser una abstracción sorprendentemente resistente.

Todo lo que alimentaba al proceso podía entrar allí.

Pero precisamente esa flexibilidad creó el siguiente problema.

Un input humano no tiene por qué venir estructurado como necesita la ingeniería.

Puede mezclar intención, requisito, restricción, decisión, preguntas abiertas y contexto en una sola página. Dos documentos pueden hablar de la misma cosa con nombres distintos. Una conversación puede contener tres conceptos que deberían evolucionar por separado.

IASI necesitaba una representación intermedia.

No un formato impuesto a quien entrega los documentos.

Una representación de **lo que IASI ha entendido**.

La idea ya había aparecido días antes como un posible adaptador entre inputs y OpenSpec. Al desaparecer la necesidad inmediata de OpenSpec, la representación intermedia dejó de ser un adaptador para otra herramienta y adquirió sentido propio.

Probamos mentalmente varios nombres.

specs/ era tentador, pero acercaba demasiado el concepto a OpenSpec y a la idea de especificación formal.

Terminamos, por ahora, en:

`definitions/`

La semántica quedó extraordinariamente sencilla:

```
inputs/      → lo que entra en IASI
definitions/ → lo que IASI ha entendido
```

Esa distinción cambió varias cosas.

Primero, desapareció la idea de una transformación uno a uno.

```
inputs/a.md
inputs/b.md → definitions/authentication.md
inputs/c.md
```

Varias fuentes pueden describir un mismo concepto.

Y al revés:

```
inputs/request.md
    → definitions/intent.md
    → definitions/scope.md
```

Un solo input puede contener varias unidades semánticas independientes.

Segundo, las definitions debían mantener trazabilidad.

No bastaba con producir una versión bonita y olvidar de dónde había salido.

Tercero, IASI no podía utilizar esa fase para inventar lo que faltaba.

Podía reformular, ordenar, consolidar y hacer explícito aquello que estuviera soportado por las fuentes.

Pero una contradicción seguía siendo una contradicción.

Y una ausencia importante seguía siendo una ausencia.

En vez de ocultarlas, **definitions/** debía poder representarlas.

Ahí empezaron a aparecer cinco naturalezas de conocimiento:

```
definition
requirement
constraint
decision
question
```

No como tipos de input.

Como tipos de conocimiento extraído.

La diferencia es esencial.

IASI no clasifica documentos.

Entiende conceptos.

Y después les da forma.

Fuentes

- `chatgpt-20260817130542-iasi-workflow-inputs-definitions.md`.
- `chatgpt-20260817135400-skill-define.md`.

title: “define no genera: reconcilia”

Este capítulo forma parte del recorrido de IASI. La narración completa continúa en el documento siguiente.

Una vez que aparecieron **definitions**/, parecía natural crear una operación capaz de producir las.

La palabra fue:

```
define
```

Al principio podía imaginarse como un generador:

```
inputs  
↓  
define  
↓  
definitions
```

Pero esa imagen duró poco.

Las definitions no eran un artefacto derivado que pudiera borrarse y regenerarse cada vez.

Eran **estado canónico editable**.

Una persona podía revisar una definition, afinar una explicación, aclarar una relación o mejorar su representación. Si la siguiente ejecución de **define** machacaba esos cambios porque no estaban literalmente en el input, el sistema estaría destruyendo conocimiento.

Así que la responsabilidad cambió.

define debía **reconciliar**.

Su contexto real era algo parecido a:

```
inputs actuales
+
definitions existentes
+
refinamientos humanos
+
templates
    ↓
    define
    ↓
estado canónico coherente
```

Esto obligó a separar razonamiento y mecánica.

El skill debía entender:

- relaciones entre fuentes;
- conceptos equivalentes;
- contradicciones;
- información ausente;
- cuándo algo es requisito o restricción;
- cuándo una pregunta ha quedado resuelta;
- cuándo una definition debe dividirse o consolidarse.

El runtime debía ocuparse de lo determinista:

- descubrir ficheros;
- fingerprints;
- gates;
- históricos;
- aplicar operaciones;
- proteger escrituras;
- registrar estado.

La frontera terminó resumiéndose así:

```
skill      → entiende y clasifica
templates  → dan forma
runtime    → valida y ejecuta
```

También apareció una fase explícita que resultó muy útil:

```
Analyse
↓
Classify
↓
Reconcile
↓
Draft
```

Classify no clasifica inputs.

Clasifica el conocimiento que ya se ha comprendido.

Por ejemplo:

```
"El sistema debe generar HTML y PDF"
  → requirement

"Para websites solo se genera HTML"
  → constraint

"Hemos decidido usar Quarto"
  → decision

"No sabemos cómo resolver EPUB"
  → question
```

Después llegaron las templates.

No debían convertirse en formularios que forzaran la realidad a entrar en cinco cajas. Debían ofrecer formas canónicas cuando la naturaleza semántica del conocimiento encajara.

Y ahí apareció otra propiedad esencial de **define**: **idempotencia semántica**.

Si nada relevante ha cambiado, volver a ejecutar **define** no debería reorganizar el mundo por capricho.

Si una persona ha refinado una definition y los inputs no invalidan ese refinamiento, debe conservarse.

Si nueva evidencia resuelve una **question**, puede reclasificarse.

Si dos definitions resultan ser el mismo concepto, pueden consolidarse.

La operación ya no era “generar documentación”.

Era mantener una base de conocimiento coherente a medida que cambia la evidencia.

Eso es una cosa bastante distinta.

Fuentes

- chatgpt-20260817135400-skill-define.md.

title: “IASI decide construirse con IASI”

Este capítulo forma parte del recorrido de IASI. La narración completa continúa en el documento siguiente.

La frase apareció antes de que estuviéramos preparados para todas sus consecuencias:

IASI se construye con IASI.

Hasta entonces IASI era la metodología que estábamos creando para trabajar sobre otros proyectos.

`iasi-graphics` había sido su primera PoC.

Pero llegó un punto en que mantener una excepción para el propio IASI empezaba a ser absurdo.

Si `inputs/`, `definitions/`, `skills`, `commands`, validaciones y `gates` eran una mejor forma de construir software, el primer proyecto que debía someterse a ellas era el propio sistema que las implementaba.

Eso introducía un problema clásico de bootstrapping.

El IASI existente había sido construido, en gran parte, antes de disponer del IASI que ahora queríamos utilizar.

No tenía sentido fingir lo contrario.

La solución conceptual fue separar dos estados.

El código actual podía actuar como **bootstrap**.

El siguiente IASI debía empezar a desarrollarse siguiendo IASI.

Para no perder el punto de partida, congelamos el estado existente con un tag.

Después apareció una rama:

```
self-hosting
```

La intención era clara:

```
IASI existente
↓
bootstrap
↓
IASI aplicado sobre sí mismo
↓
IASI siguiente
```

El código anterior no se convertía en basura.

Seguía siendo implementación, referencia, evidencia y fuente de conocimiento.

Pero dejaba de ser automáticamente la especificación del futuro.

Eso es una distinción importante.

Cuando un sistema se reconstruye a sí mismo existe una tentación enorme de describir como requisito todo aquello que ya hace.

Pero IASI había nacido, precisamente, para cuestionar las soluciones actuales.

La implementación bootstrap podía enseñar.

No podía mandar.

Los primeros inputs del self-hosting empezaron a describir el propio mecanismo de definitions y define.

La idea era atractiva porque convertía el desarrollo en una prueba continua:

Si IASI no puede construir IASI, tenemos un problema bastante más interesante que un test fallido.

Sin embargo, casi inmediatamente descubrimos otra cosa.

La recursión conceptual era correcta.

Nuestra capacidad para mantenerla entera en la cabeza no lo era.

Fuentes

- chatgpt-20260817130542-iasi-workflow-inputs-definitions.md.
- Conversación de 2026-08-17 sobre el tag de bootstrap y la rama self-hosting.
- Memoria de trabajo de IASI.

title: “Nos perdimos y paramos”

Este capítulo forma parte del recorrido de IASI. La narración completa continúa en el documento siguiente.

El self-hosting produjo entusiasmo durante unos minutos.

También produjo confusión.

Habíamos congelado el IASI existente con un tag. Habíamos creado la idea de una rama **self-hosting**. Teníamos inputs sobre **definitions** y el skill **define**. Empezamos a hablar de procesarlos, generar definitions, reconstruir el programa y usar el viejo IASI como bootstrap.

Todo era razonable por separado.

Junto, dejó de ser comprensible.

La frase que cortó la secuencia fue:

Me estoy perdiendo.

Intentamos simplificar.

Enumeramos dónde estábamos.

Volvimos a explicar que todavía no habíamos recreado IASI.

Apareció la rama.

Volvimos a preguntar qué debía hacer Copilot.

Y finalmente:

Vamos a parar aquí.

Ese momento merece una entrada del Journey precisamente porque no produjo código.

Produjo información sobre la metodología.

Los Sistemas Inteligentes eliminan muchísimas fricciones que antes actuaban como pausas naturales.

Una pregunta obtiene respuesta inmediatamente. Una arquitectura puede extenderse en segundos. Mientras un agente implementa una cosa, podemos abrir otra línea de pensamiento. Los bloqueos desaparecen y con ellos desaparecen también intervalos en los que el humano solía recuperar contexto.

La velocidad técnica aumenta.

La capacidad humana para comprender, decidir y asimilar no aumenta en la misma proporción.

Ese desfase ya había aparecido como principio metodológico de IASI:

No confundir disponibilidad de la IA con disponibilidad humana.

El episodio de self-hosting lo convirtió en experiencia directa.

No había un error técnico.

Había demasiados cambios de nivel conceptual seguidos:

```
bootstrap
→ tag
→ branch
→ inputs
→ definitions
→ skill
→ command
→ reconstrucción
→ Copilot
```

El sistema de ideas seguía avanzando.

El modelo mental humano empezaba a quedarse atrás.

La respuesta correcta no era pedir a la IA que explicara todavía más deprisa.

Era parar.

Esto introduce una clase de gate que no habíamos modelado en el workflow.

Tenemos gates técnicos:

```
validated  
planned  
verified
```

Pero puede existir otro límite:

```
¿el humano sigue entendiendo lo suficiente  
como para tomar la siguiente decisión?
```

No es fácilmente automatizable y quizá nunca deba serlo.

Pero sí debe formar parte de la metodología.

La velocidad sostenible de un sistema humano-IA no es la velocidad máxima de los agentes.

Es la velocidad a la que el humano puede mantener comprensión suficiente para seguir siendo responsable de las decisiones.

Lo que quedó

Parar no interrumpió el proceso.

Fue parte del proceso.

Y el hecho de que esta entrada exista es exactamente la razón por la que necesitábamos un Journey.

El book futuro podrá explicar limpiamente el self-hosting.

El Journey debe conservar que, cuando llegamos por primera vez allí, nos hicimos un lío.

Fuentes

- Conversación de 2026-08-17 sobre self-hosting de IASI.
- Memoria de IASI sobre límites humanos, fatiga cognitiva y velocidad sostenible.

title: “El Journey empieza a escribirse”

Este capítulo forma parte del recorrido de IASI. La narración completa continúa en el documento siguiente.

El Journey llevaba varios días existiendo como idea antes de tener documentos.

Eso también era coherente con su naturaleza.

Primero apareció la necesidad de conservar conversaciones. Después la distinción entre raw y treated. Luego la separación entre Journey, decisiones, libros y artefactos. Más tarde empezamos a señalar explícitamente frases y episodios como “esto es Journey”.

Pero todavía no lo habíamos escrito.

La pregunta que finalmente puso el repositorio en movimiento fue sencilla:

¿Te apetece empezar con el Journey?

La primera discusión no fue sobre Quarto.

Fue sobre el enfoque.

No queríamos un diario:

```
17 de agosto  
hoy hicimos...  
después hicimos...
```

La fecha importaba, pero no debía ser la unidad narrativa.

La unidad debía ser **el episodio que cambia algo**.

Una idea aparece, se discute, tropieza con la realidad y deja una consecuencia.

Eso permite que una entrada dure una hora o atravesase varios días.

También decidimos algo más importante que la estructura:

No reescribir la historia para que parezca que sabíamos desde el principio dónde íbamos.

Si **definitions**/ surgió después de varios intentos, el Journey debía contarlos así.

Si OpenSpec parecía necesario y después dejó de serlo, ambas cosas debían coexistir.

Si una propuesta vino primero de la IA y Javier la corrigió, eso también forma parte del proceso.

Si nos perdimos, se escribe que nos perdimos.

Después sí llegó Quarto.

El Journey seguirá siendo un proyecto capaz de producir HTML y PDF. La lectura principal puede ser viva y web, pero periódicamente podrá cristalizar en una edición imprimible.

Eso introduce una tensión interesante.

El contenido debe seguir siendo histórico e inmutable en su sentido.

La publicación, en cambio, puede evolucionar: estilo, navegación, índices, enlaces, selección editorial.

Como ocurría con raw y treated, volvemos a separar fuente y representación.

Y finalmente llegó el gesto que convierte la idea en proyecto:

entregar los chats acumulados y decir:

Empieza a crear documentos de Journey. Todos los que quieras.

Esta primera tanda no pretende cerrar la historia.

Hace algo más útil.

Crea un primer mapa narrativo de lo que ya ocurrió y deja un formato con el que podamos seguir trabajando.

A partir de aquí, el Journey ya no es una promesa en otro repositorio.

Ha empezado.

Fuentes

- Conversación de 2026-08-17 sobre el enfoque y estructura de **iasi-journey**.
- Chats entregados para la primera construcción del Journey.
- Memoria de trabajo de IASI.